Assignment: Data Structure and System Design

Name: Dhanyatha S
Email: dhanyatha237.y@gmail.com

1. LRU Cache Implementation

Problem: Consider a browser that can only hold 3 tabs in memory at once, and opened 4th tab, it closes the least recently used which means whichever tab was not used in longest of time will be closed.

Requirements:

1. If the tab opened within the slot range get the key (tab[i]) and return the value if found, otherwise return -1 and must mark as recently used.
2. Add or update the next tab session
3. If max slot capacity has reached it must evict the tab which has not used for longer and marking Least Recently Used.
4. *IMPORTANT* Both Operation must run in O(1) time Complexity – in constant time

Solution:

Since we need to achieve the O(1) time complexity, that means constantly Searching for keys to know the max capacity, and tracing and traversing back and forth by shifting the nodes and remove in the middle which takes worst time complexity O(n) (searching).

The Optimized and best solution would be using Hash map and Doubly Linked-List together.

<u>Hash-Map:</u> helps to look-up instantly via map, key pointing to the node in the Linked List this give O(1) for look up (<u>no Searching</u>).

<u>Doubly Linked List</u>: Will Track the order of use by maintaining it of used tabs and remove it from middle at same time.

Head Tail

[Most Recently Used] ……..[Least Recently Used]

Head end holds most recently used tabs so as tail LRU, because each node has both prev and next pointer. We can remove any node from middle of the list in O(1) (no Shifting)

[Head]

|

[Null | Data | Next] ⇄ [Prev | Data | Next] …..

On get(key): will find node instantly via map, move it to the front

On put(key): adding that key to front, if over capacity remove from tail.

Neither Structure alone achieve O(1) for everything

a. HashMap has no Ordering
b. Linked List alone requires O(n) for Searching for a key

Together, the HashMap handles fast lookup and Linked List (Double) handles ordering and eviction.

Code: Github Link

2. Event Scheduling

Concept of Managing "Time, Sequence, Priority of tasks (events) to maximize system efficiency."

Problem: Need to design a feasible scheduling pipeline for a meeting event (Meeting Platform) so that it does not overlap with the person's schedule who will be attending all the meeting , also manage the minimum number of rooms required for all the meetings happening simultaneously.

Requirements:

1. Given the Events with start time and end time, manage the events so that a person can attend without overlap, then return true. Otherwise, return False.

2. Assign the minimum number of rooms for all the meetings scheduled without overlapping, and return it.

Solution:

can_attend_all(events):

1. Picking a person and sort by start_time first, then check if next meeting starts before the previous one ends. If yes there is overlapping,
2. If (end_time == next start_time) no overlapping a person can attend the next meeting.

Method: sort the leading, then linear traversal to check the neighbours. Time Complexity will take O(n log n).

min_room_required(events):

1. By using a heap to store the end_time of the currently running meeting and repeatedly updating. If, each time a new meeting starts, the earliest ending has already finished, reuse the room (pop from the heap) for next meeting. if not we need a new room to start the next concurrent meeting. The heap size at the peak is the min_room_required.
2. For 'n' events we require O(n log n) sorting and each event does heap push/pop with O(log n) so 'n' = O(n log n).

Code: Github Link

Final Discussion and Analysis

1. Time and Space Complexity

"LRU Cache Implementation":

get(key)	Time O(1)	Space O(1)	HashMap jumps to the node instantly. Linked list moves it to front in O(1), 2 pointers.
put(key)	Time O(1)	Space O(1)	Insert node at front + add to HashMap. If over capacity, remove tail node + delete from HashMap. All pointer operations. So, no loops.
_remove(node)	Time O(1)	Space O(1)	Rewires prev.next and next.prev. Fixed number of operations regardless of cache size.
Overall Storage	Space O(1)		HashMap holds at most *capacity* entries. Linked list holds at most *capacity* nodes. Total memory is bounded and fixed.

The tab walks through every function one by one. The key insight is that all LRU cache operations are O(1) because they are purely pointer manipulation, the linked list never loops, and the HashMap never searches.

"Event Scheduler"

can_attend_all(events)	Time O(n log n)	Space O(n)	Sort dominates. Then one linear pass to check neighbours. Space is for the sorted copy of events.
min_rooms_required(events)	Time O(n log n)	Space O(n)	Sort is O(n log n). Each event does one heap push/pop, each of them are O(log n), so n events = O(n log n) total. Heap holds at most n end times.

Why O(n log n) is Optimal ?: To know if events overlap we must compare them in time order, and sorting is required to establish that order. Comparison-based sorting has a proven lower bound of O(n log n), so this is optimal.

2. Trade-Off

<u>LRU Cache Implementation:</u>

HeapMap alone can't do at O(1) because: Lookup is O(1) which is great, but it can not track the key which is impossible to it and no concept of last used and it could not evict any key because it must need to scan all the entries.

Linked List can't do at O(1) alone because: It tracks order of key at O(1) however, it must lookup – must scan from head to tail which requires O(n), and will evict at O(1).

So, what works well is HashMap + doubly linked list together

Lookup: O(1) - HashMap jumps directly to the node in the list

Reorder: O(1) - doubly linked means we can detach and reattach any node with 4 pointer changes. A singly linked list can't do this because we need to update the previous node's pointer, and we don't have a reference to it.

Evict LRU: O(1) - the node just before the dummy tail is always the least recently used. Remove it and delete its key from the HashMap.

3. Future Proofing

Assigning specific room names to events

The current code just counts rooms. To assign names like "Room A", "Room B", we make one small change, instead of just tracking end times in the heap, we also track which room name is free.

With these 3 steps

1. Start with a pool of available room names like a queue ["Room A", "Room B", "Room C", ...etc,.]. The queue grows as needed.
2. Heap stores (end_time, room_name) pairs, when we pop a finished meeting, we get back the room name it was using and return that name to the pool.
3. Record the event details, store event -> room_name in a dictionary so you can look up which room any event was given.

4. Concurrency: Making LRUCache Thread-Safe

The current implementation is NOT thread-safe. If two threads call get() or put() at the same time, they can corrupt the linked list or HashMap (e.g., two threads both try to evict the LRU node simultaneously).

Fixes:

1. Coarse-grained lock (simplest): Wrap the entire get() and put() body in a threading.Lock(). Simple and correct, but only one thread can use the cache at a time.

2. Read-write lock (better throughput): Allow multiple concurrent readers (get), but get() also mutates order, so in a strict LRU this still needs a write lock on every access. A read-friendly variant is an LFU or a non-ordering cache for reads.

3. Concurrent data structure (advanced): Use concurrent hash maps (e.g. from concurrent.futures or Java's ConcurrentHashMap) with fine-grained per-segment locking to reduce contention. Often used in production systems.

Simplest correct Python implementation:

```

import threading

self.lock = threading.Lock()

# then in get() and put():

with self.lock: ... existing logic ...

```